

1. Introduction

This project aimed to classify arrhythmia from electrocardiogram (ECG) data. The dataset contained various features extracted from ECG signals, and the goal was to build and evaluate several machine learning and deep learning models to accurately identify different types of arrhythmia.

2. Methodology

2.1 Data Loading and Preprocessing

- The arrhythmia.csv dataset was loaded using pandas.
- Missing values, represented by '?', were replaced with np.nan and then imputed using the mean strategy.
- Features were scaled using StandardScaler.
- Principal Component Analysis (PCA) was applied to reduce dimensionality to 100 components.
- To address class imbalance, the SMOTE (Synthetic Minority Over-sampling Technique) algorithm was applied to the PCA-transformed data.
- The data was then split into training and testing sets (80% train, 20% test) with stratification.
- Labels for Keras/TabNet models were indexed (0-based) for compatibility.
- Data was reshaped for CNN and LSTM models to accommodate their input requirements.

2.2 Model Training and Evaluation

Several machine learning and deep learning models were trained and evaluated on the preprocessed data. The models included:

Scikit-learn Models:

- Tuned Support Vector Machine (SVM)
- Random Forest Classifier
- Logistic Regression
- K-Nearest Neighbors (KNN)
- LightGBM

- AdaBoost Classifier
- Decision Tree Classifier

Keras/TensorFlow Models:

- Multi-Layer Perceptron (MLP)
- TabNet
- Convolutional Neural Network (CNN)
- Long Short-Term Memory (LSTM)

An **Ensemble Model** using VotingClassifier with Tuned SVM and Random Forest Classifier was also implemented.

Each model's performance was assessed using:

- **Accuracy**
- **Macro Avg F1-score**
- **Weighted Avg F1-score**

2.3 Visualizations

- A bar plot was generated to compare the Accuracy, Macro Avg F1, and Weighted Avg F1 scores across all trained models.
- Confusion matrices were generated for the top 4 performing models to provide a detailed breakdown of their classification performance for each arrhythmia class.

3. Results and Discussion

3.1 Model Performance Summary

The model_performance_summary.csv file provides a tabular overview of all models' performance. The top models, based on accuracy, were generally:

Model	Accuracy	Macro Avg F1	Weighted Avg F1
Ensemble (SVM+RF)	0.9984	0.9984	0.9984
RandomForestClassifier	0.9953	0.9953	0.9953

Model	Accuracy	Macro Avg F1	Weighted Avg F1
Tuned MLP	0.9937	0.9936	0.9936
Tuned SVM	0.9890	0.9887	0.9887

The **Ensemble (SVM+RF)** model achieved the highest accuracy and F1-scores, demonstrating the benefit of combining multiple strong classifiers. RandomForestClassifier and Tuned MLP also showed excellent performance, indicating their robustness for this classification task.

3.2 Visual Analysis

- **Model Performance Comparison Bar Plot (model_performance_comparison.png):** This visualization clearly illustrates the relative performance of each model across the three metrics. It confirms that the ensemble and tree-based models (RandomForest, LightGBM) generally outperformed the single traditional ML models and some deep learning architectures.
- **Confusion Matrices (e.g., Confusion_Matrix_for_Ensemble_(SVM+RF).png):** The confusion matrices provide granular insights into where models excel and where they make errors. For the top-performing models, these matrices show high values along the diagonal, indicating correct classifications. Any off-diagonal values highlight specific classes that are misclassified, which can inform further model refinement or data collection efforts.

4. Conclusion

This project successfully applied various machine learning and deep learning techniques for arrhythmia classification. The ensemble model (SVM+RandomForest) demonstrated superior performance, closely followed by RandomForest and Tuned MLP. The detailed performance metrics and visualizations provide a solid foundation for understanding the strengths and weaknesses of each approach. Further work could involve exploring more advanced deep learning architectures, optimizing hyperparameters more extensively, or incorporating more sophisticated feature engineering techniques.